UNIVERSITÉ SORBONNE PARIS NORD
IUT DE VILLETANEUSE
Département Informatique

S104 : Création d'une base de données

RAPPORT DE SYNTHÈSE :

Groupe MARS
Anass Fathi

Destinataire : M. Floquet

Sous la direction de : M Laurent Audibert

Date de rendu : 21 Janvier 2026

Table des Matières

I - Script manuel de création de la base de données .....…….. 3 page

II – Modélisation et script de création “ avec AGL” ..... 4 page

A. Illustration comparative cours d’une association fonctionnelle…. 4 page

B. Illustration comparative cours d’une association maillé .....….. 5 page

C. Modèle physique de données réalisé avec L’AGL …..……6 page

D. Script SQL de création des tables généré automatiquement par L’AGL. 6 page

E. Discussion sur les différences entre les scripts produits manuellement et généré 10 page

III – Peuplement des tables ……..... 10 page

A. Création de la base de données ...…….. 10 page

B. Peuplement de la table : REGIONS .....……..... 11 page

C. Peuplement de la table : DEPARTEMENTS ..……... 11 page

D. Peuplement de la table : COMMUNES …… 12 page

E. Peuplement de la table : SITES_...……..... 12 page

F. Peuplement de la table : ANALYSES.....….……..... 13 page

G. Peuplement de la table : EVENEMENTS .....……..... 13 page

IV – Conclusion Peuplement et Synthèse du rapport ……..……... 14 page

I. Script manuel de création de la base de données

```
CREATE TABLE REGIONS (
     code_regions VARCHAR(10) PRIMARY KEY,
     nom_regions VARCHAR(50) NOT NULL
);

CREATE TABLE DEPARTEMENTS (
     code_departement VARCHAR(10) PRIMARY KEY,
     region VARCHAR(10) NOT NULL REFERENCES REGIONS(code_regions),
     nom_departement VARCHAR(50) NOT NULL
);


CREATE TABLE COMMUNES (
     code_commune VARCHAR(10) PRIMARY KEY,
     departement VARCHAR(10) NOT NULL REFERENCES DEPARTEMENTS(code_departement),
     nom_commune VARCHAR(50) NOT NULL
);


CREATE TABLE SITES (
    idSite VARCHAR(100) PRIMARY KEY,
    nom_site VARCHAR(200) NOT NULL,
    code_commune VARCHAR(10) NOT NULL REFERENCES COMMUNES(code_commune),
    date_Declaration Date,
    typeEau VARCHAR(100),
    longitude DECIMAL(9,6),
    latitude DECIMAL(9,6)
);

 CREATE TABLE EVENEMENTS (
      idEvenement INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
      idSite VARCHAR(100) NOT NULL REFERENCES SITES(idSite),
      evenement VARCHAR(500),
      debut DATE,
      fin DATE,
      mesure VARCHAR(500)
);

CREATE TABLE ANALYSES (
     idAnalyse INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
     idSite VARCHAR(100) NOT NULL REFERENCES SITES(idSite),
     datePrelevement DATE NOT NULL,
     enterocoques INTEGER,
     escherichia INTEGER
);
```

II. Modélisation et script de création “ avec AGL”

A. Illustration comparative cours d’une association fonctionnelle :

Modèle physique utilisée : AGL , PGModeler

Définition d'un type association fonctionnelle :

Une association fonctionnelle est une association entre deux types-entités A et B telle que la cardinalité maximale d’au moins l’une d’elle soit égale à 1. Cela signifie qu’une occurrence de l’un des types-entités peut être associée à au plus une occurrence de l’autre type-entité.

Pour illustrer le type association fonctionnelle je me suis basée sur la partie de la modélisation entités-association de la base de données ci-contre.

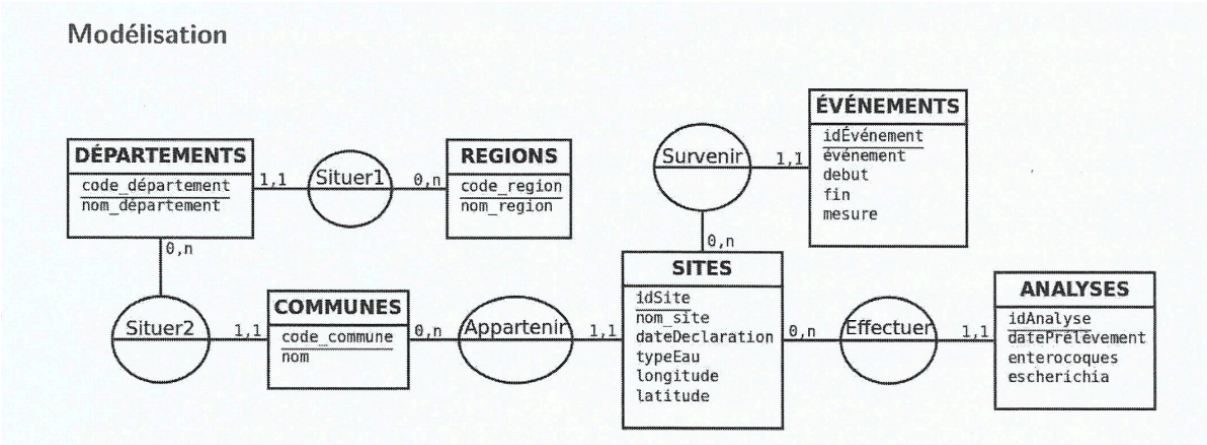
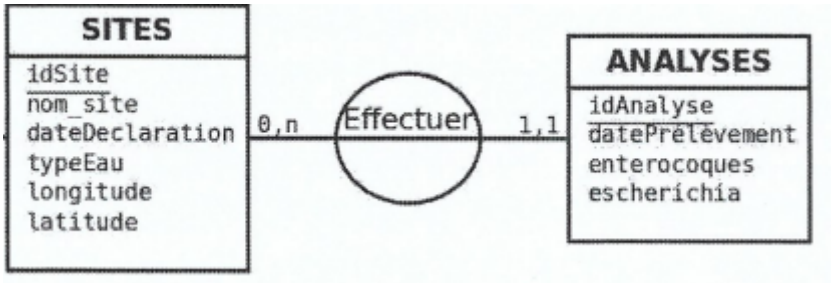


Fig. 1 : Modélisation entités-associations de la base de données

Plus précisément pour notre cas d’association fonctionnelle , prenons le cas suivant :



sens de lecture :

ANALYSES (1,1) ======> Effectuer ======> SITES (0,n)

Dans le cas de notre AGL :

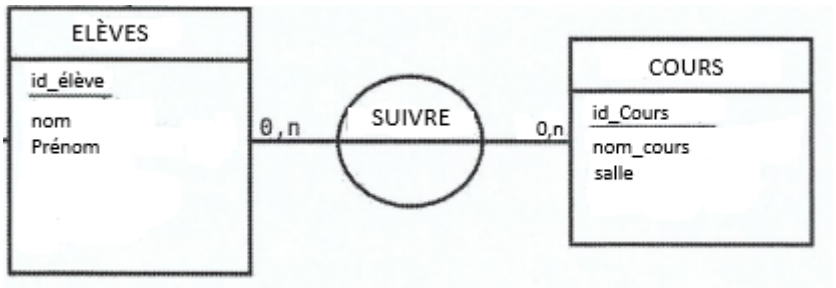
Le cas choisi se lit de la façon suivante : une analyse est effectuée sur un unique site ,nous avons donc une cardinalité (1,1) du côté du type-Entité ANALYSES . De plus ,un site peut effectuer aucune à plusieurs analyses ,par cela nous avons une cardinalité (0,n) du côté du type-Entité SITES . Concordant ainsi à la définition plutôt rédiger, nous avons bien une cardinalité maximale égale à 1 du côté ANALYSES et une cardinalité maximale égale à n du côté SITES . Cette association est donc fonctionnelle, car une occurrence du type-entité ANALYSES (côté « un ») est associée à au plus une occurrence du type-entité SITES .

B. Illustration comparative cours d'une association maillé

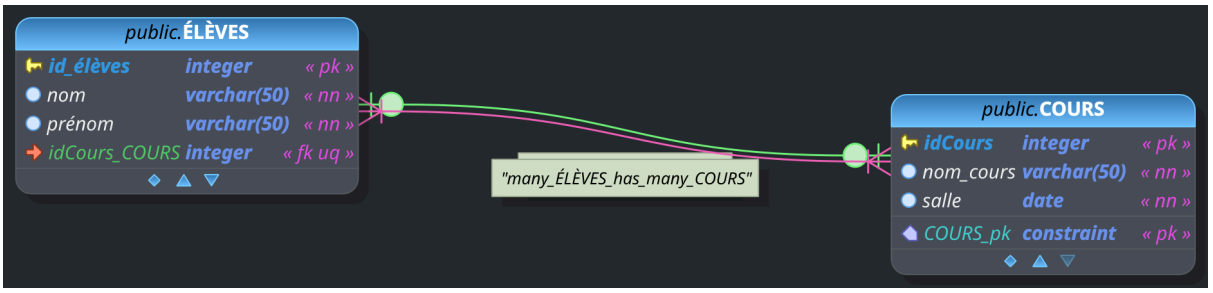
Définition d'un type association maillé :

Une association maillée est une association entre deux types-entités A et B telle que les cardinalités maximales des deux côtés soient n . Cela signifie qu'une occurrence de l'un des types-entités peut être associée à plusieurs occurrences de l'autre type-entité, et réciproquement.

Plus précisément pour notre cas d'association maillé , prenons le cas suivant :

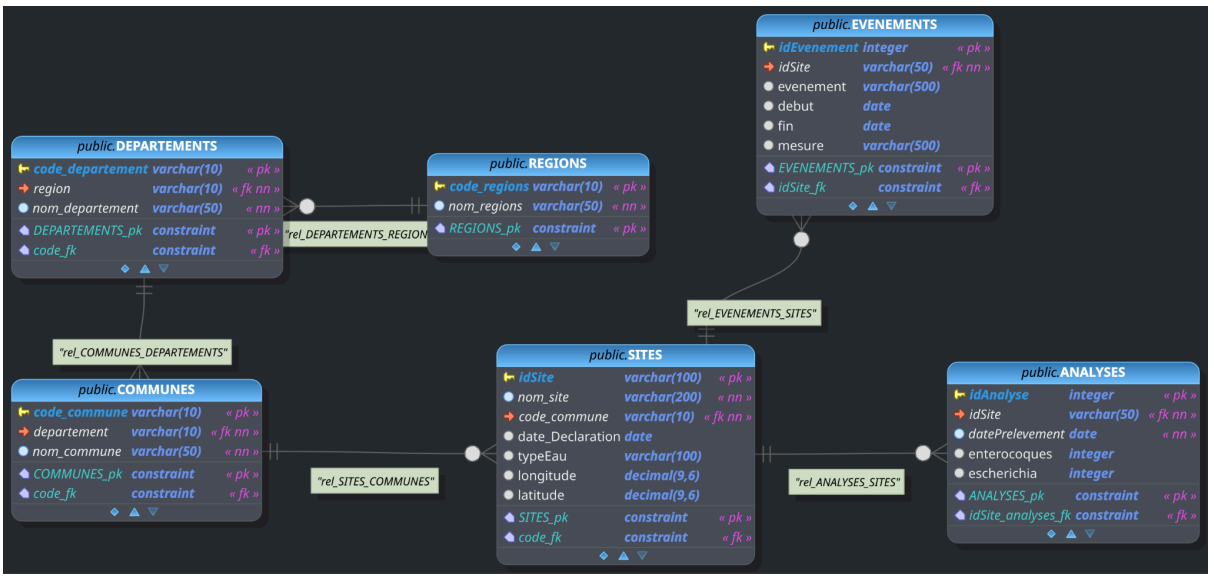


ELÈVES (0,n) ======> Suivre ======> COURS (0,n)



Le cas choisi se lit de la façon suivante : un élève peut suivre zéro à plusieurs cours, nous avons donc une cardinalité (0,n) du côté du type-Entité ÉLÈVES . De plus, un cours peut être suivi par zéro à plusieurs élèves . Nous avons donc une cardinalité (0,n) du côté du type-Entité COURS . Concordant ainsi à la définition plutôt rédiger, nous avons bien une cardinalité maximale égale à n du côté ÉLEVÉS et une cardinalité maximale égale à n du côté COURS. Cette association est donc maillé, car zéro à plusieurs occurrence du type-entité ÉLÈVES peut être associée à zéro a plusieur occurrence du type-entité COURS.

C. Modèle physique de données réalisé avec L'AGL :



D. Script SQL de création des tables généré automatiquement par L'AGL :

```
-- ** Database generated with pgModeler (PostgreSQL Database Modeler).
-- ** pgModeler version: 1.2.2
-- ** PostgreSQL version: 18.0
-- ** Project Site: pgmodeler.io
-- ** Model Author: ---
```

```sql
-- ** Database creation must be performed outside a multi lined SQL
file.
-- ** These commands were put in this file only as a convenience.

-- object: new_database | type: DATABASE --
-- DROP DATABASE IF EXISTS new_database;
CREATE DATABASE new_database;
-- ddl-end --


SET search_path TO pg_catalog,public;
-- ddl-end --

-- object: public."SITES" | type: TABLE --
-- DROP TABLE IF EXISTS public."SITES" CASCADE;
CREATE TABLE public."SITES" (
    "idSite" varchar(100) NOT NULL,
    nom_site varchar(200) NOT NULL,
    code_commune varchar(10) NOT NULL,
    "date_Declaration" date,
    "typeEau" varchar(100),
    longitude decimal(9,6),
    latitude decimal(9,6),
    CONSTRAINT "SITES_pk" PRIMARY KEY ("idSite")
);
-- ddl-end --
ALTER TABLE public."SITES" OWNER TO postgres;
-- ddl-end --

-- object: public."ANALYSES" | type: TABLE --
-- DROP TABLE IF EXISTS public."ANALYSES" CASCADE;
CREATE TABLE public."ANALYSES" (
    "idAnalyse" integer NOT NULL,
    "idSite" varchar(100) NOT NULL,
    "datePrelevement" date NOT NULL,
    enterocoques integer,
    escherichia integer,
    CONSTRAINT "ANALYSES_pk" PRIMARY KEY ("idAnalyse")
);
-- ddl-end --
ALTER TABLE public."ANALYSES" OWNER TO postgres;
-- ddl-end --

-- object: public."REGIONS" | type: TABLE --
-- DROP TABLE IF EXISTS public."REGIONS" CASCADE;
CREATE TABLE public."REGIONS" (
```

```sql
    code_regions varchar(10) NOT NULL,
    nom_regions varchar(50) NOT NULL,
    CONSTRAINT "REGIONS_pk" PRIMARY KEY (code_regions)
);
-- ddl-end --
ALTER TABLE public."REGIONS" OWNER TO postgres;
-- ddl-end --

-- object: public."DEPARTEMENTS" | type: TABLE --
-- DROP TABLE IF EXISTS public."DEPARTEMENTS" CASCADE;
CREATE TABLE public."DEPARTEMENTS" (
    code_departement varchar(10) NOT NULL,
    region varchar(10) NOT NULL,
    nom_departement varchar(50) NOT NULL,
    CONSTRAINT "DEPARTEMENTS_pk" PRIMARY KEY (code_departement)
);
-- ddl-end --
ALTER TABLE public."DEPARTEMENTS" OWNER TO postgres;
-- ddl-end --

-- object: public."COMMUNES" | type: TABLE --
-- DROP TABLE IF EXISTS public."COMMUNES" CASCADE;
CREATE TABLE public."COMMUNES" (
    code_commune varchar(10) NOT NULL,
    departement varchar(10) NOT NULL,
    nom_commune varchar(50) NOT NULL,
    CONSTRAINT "COMMUNES_pk" PRIMARY KEY (code_commune)
);
-- ddl-end --
ALTER TABLE public."COMMUNES" OWNER TO postgres;
-- ddl-end --

-- object: public."EVENEMENTS" | type: TABLE --
-- DROP TABLE IF EXISTS public."EVENEMENTS" CASCADE;
CREATE TABLE public."EVENEMENTS" (
    "idEvenement" integer NOT NULL,
    "idSite" varchar(100) NOT NULL,
    evenement varchar(500),
    debut date,
    fin date,
    mesure varchar(500),
    CONSTRAINT "EVENEMENTS_pk" PRIMARY KEY ("idEvenement")
);
-- ddl-end --
ALTER TABLE public."EVENEMENTS" OWNER TO postgres;
-- ddl-end --
```

```sql
-- object: code_fk | type: CONSTRAINT --
-- ALTER TABLE public."SITES" DROP CONSTRAINT IF EXISTS code_fk CASCADE;
ALTER TABLE public."SITES" ADD CONSTRAINT code_fk FOREIGN KEY
(code_commune)
REFERENCES public."COMMUNES" (code_commune) MATCH SIMPLE
ON DELETE NO ACTION ON UPDATE NO ACTION;
-- ddl-end --

-- object: "idSite_analyses_fk" | type: CONSTRAINT --
-- ALTER TABLE public."ANALYSES" DROP CONSTRAINT IF EXISTS
"idSite_analyses_fk" CASCADE;
ALTER TABLE public."ANALYSES" ADD CONSTRAINT "idSite_analyses_fk"
FOREIGN KEY ("idSite")
REFERENCES public."SITES" ("idSite") MATCH SIMPLE
ON DELETE NO ACTION ON UPDATE NO ACTION;
-- ddl-end --

-- object: code_fk | type: CONSTRAINT --
-- ALTER TABLE public."DEPARTEMENTS" DROP CONSTRAINT IF EXISTS code_fk
CASCADE;
ALTER TABLE public."DEPARTEMENTS" ADD CONSTRAINT code_fk FOREIGN KEY
(region)
REFERENCES public."REGIONS" (code_regions) MATCH SIMPLE
ON DELETE SET DEFAULT ON UPDATE NO ACTION;
-- ddl-end --

-- object: code_fk | type: CONSTRAINT --
-- ALTER TABLE public."COMMUNES" DROP CONSTRAINT IF EXISTS code_fk
CASCADE;
ALTER TABLE public."COMMUNES" ADD CONSTRAINT code_fk FOREIGN KEY
(departement)
REFERENCES public."DEPARTEMENTS" (code_departement) MATCH SIMPLE
ON DELETE NO ACTION ON UPDATE NO ACTION;
-- ddl-end --

-- object: "idSite_fk" | type: CONSTRAINT --
-- ALTER TABLE public."EVENEMENTS" DROP CONSTRAINT IF EXISTS "idSite_fk"
CASCADE;
ALTER TABLE public."EVENEMENTS" ADD CONSTRAINT "idSite_fk" FOREIGN KEY
("idSite")
REFERENCES public."SITES" ("idSite") MATCH SIMPLE
ON DELETE NO ACTION ON UPDATE NO ACTION;
-- ddl-end --
```

E. Discussion sur les différences entre les scripts produits manuellement et Généré

L ' analyse entre le script SQL rédigé par mes soins lors de la question 2.1 et celui généré automatiquement par l'AGL PGModeler révèle plusieurs points différents importants .

En effet, si mon script adopte une clarté et privilégie la compréhension de tous, celui de PGModeler lui se base sur une structure plus complexe à comprendre et privilégie la solidité de part une structure méthodique approchant des attendus finaux de bases de données plus complexes. Les différences primaires de ces deux scripts se différencient tout d'abord par la création d'une en-tête de la part de PGModeler précisant toutes les informations concernant la version et le modèle utilisé pour cette génération. Les différences se présentent aussi par l'omission de la clause `GENERATED ALWAYS AS IDENTITY` qui a pour but de générer les clés pour la gestion automatisée des identifiants.

De plus, PGModeler inclut aussi les commandes pour la création de la base de données ainsi que des commandes pour définir les droits utilisateurs sur celle-ci, notamment avec `OWNER TO` et `ALTER TABLE`. Enfin, le dernier point relevé concerne les définitions des contraintes pour chacune des tables. Celles-ci sont faites dans mon script dans la table elle-même . PGModeler lui les structure à la fin en dehors de la création des tables. Il sépare ainsi la structure des tables de leurs relations avec les différentes tables présentes dans la base de données.

III. Peuplement des tables

A. Création de la base de données

```
anass=> CREATE DATABASE sae104_anass_f;

anass=> \c sae104_anass_f

Vous êtes maintenant connecté à la base de données « sae104_anass_f » en
tant qu'utilisateur « anass ».

sae104_anass_f=> \i
/home/anass/intro_systeme/R104/SAE104/scripts/figure_1_script.sql
```

B. Peuplement de la table : REGIONS

```
sae104_anass_f=> CREATE TEMP TABLE temp_regions (
    col1 TEXT, col2 TEXT, col3 TEXT, col4 TEXT
);

sae104_anass_f=> \copy temp_regions FROM
'/home/anass/intro_systeme/R104/SAE104/donnee/departements-france.csv'
WITH (FORMAT csv,HEADER true ,DELIMITER ',',ENCODING 'LATIN1');

sae104_anass_f=>INSERT INTO REGIONS ("code_regions", "nom_regions")
SELECT DISTINCT
    col3,
    col4
FROM temp_regions
WHERE col3 IS NOT NULL;
```

C. Peuplement de la table : DEPARTEMENTS

```
sae104_anass_f=> CREATE TEMP TABLE temp_departements (
    col1 TEXT, col2 TEXT, col3 TEXT, col4 TEXT
);

sae104_anass_f=> \copy temp_departements FROM
'/home/anass/intro_systeme/R104/SAE104/donnee/departements-france.csv'
WITH (FORMAT csv,HEADER true ,DELIMITER ',',ENCODING 'LATIN1');

sae104_anass_f=> INSERT INTO DEPARTEMENTS ("code_departement",
"region","nom_departement")
SELECT DISTINCT
    LTRIM(col1,'0'),
    col3,
    col2
FROM temp_departements
WHERE col1 IS NOT NULL;
```

D. Peuplement de la table : COMMUNES

```
sae104_anass_f=> CREATE TEMP TABLE temp_communes (
    col1 TEXT, col2 TEXT, col3 TEXT, col4 TEXT, col5 TEXT,
    col6 TEXT, col7 TEXT, col8 TEXT, col9 TEXT, col10 TEXT,
    col11 TEXT, col12 TEXT, col13 TEXT
);

sae104_anass_f=> \copy temp_communes FROM
'/home/anass/intro_systeme/R104/SAE104/donnee/liste-des-sites-de-baignad
e-saison-balneaire-2024.csv' WITH (FORMAT csv,HEADER true ,DELIMITER
';',ENCODING 'LATIN1');

sae104_anass_f=> INSERT INTO COMMUNES ("code_commune",
"departement","nom_commune")
SELECT DISTINCT
    col8,
    LTRIM(col3, '0'), -- On enlève le '0' qui sont devant les chiffres
    col9
FROM temp_communes
WHERE col8 IS NOT NULL;
```

E. Peuplement de la table : SITES

```
sae104_anass_f=> CREATE TEMP TABLE temp_sites (
    col1 TEXT, col2 TEXT, col3 TEXT, col4 TEXT, col5 TEXT,
    col6 TEXT, col7 TEXT, col8 TEXT, col9 TEXT, col10 TEXT,
    col11 TEXT, col12 TEXT, col13 TEXT
);

sae104_anass_f=> \copy temp_sites FROM
'/home/anass/intro_systeme/R104/SAE104/donnee/liste-des-sites-de-baignad
e-saison-balneaire-2024.csv' WITH (FORMAT csv,HEADER true ,DELIMITER
';',ENCODING 'LATIN1');

sae104_anass_f=> INSERT INTO SITES
(idSite,nom_site,code_commune,typeEau,longitude,latitude,date_Declaratio
n)
SELECT
   col4,col7,col8,col11,
   NULLIF(REPLACE(col12, ',', '.'), ''):::DECIMAL,
   NULLIF(REPLACE(col13, ',', '.'), ''):::DECIMAL,
   NULLIF(col10, ''):::DATE
   FROM temp_sites;
```

F. Peuplement de la table : ANALYSES

```
sae104_anass_f=> CREATE TEMP TABLE temp_analyses (
    col1 TEXT, col2 TEXT, col3 TEXT, col4 TEXT, col5 TEXT,
    col6 TEXT, col7 TEXT, col8 TEXT, col9 TEXT, col10 TEXT,
    col11 TEXT
);

sae104_anass_f=> \copy temp_analyses FROM
'/home/anass/intro_systeme/R104/SAE104/donnee/saison-balneaire-2024-resu
ltats-danalyses.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'LATIN1');

sae104_anass_f=> INSERT INTO ANALYSES (idSite, datePrelevement,
enterocoques, escherichia)
SELECT
    col4,                       -- idSite
    col5::DATE,                 -- datePrelevement
    NULLIF(col6, ''):::INTEGER, -- enterocoques
    NULLIF(col7, ''):::INTEGER  -- escherichia
FROM temp_analyses;
```

G. Peuplement de la table : EVENEMENTS

```
sae104_anass_f=> CREATE TEMP TABLE temp_evenements (
    col1 TEXT, col2 TEXT, col3 TEXT, col4 TEXT,
    col5 TEXT, col6 TEXT, col7 TEXT, col8 TEXT );

sae104_anass_f=> \copy temp_evenements FROM
'/home/anass/intro_systeme/R104/SAE104/donnee/saison-balneaire-2024-info
rmations-sur-la-saison.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'LATIN1');

sae104_anass_f=> INSERT INTO EVENEMENTS (idSite,evenement,debut, fin,
mesure)
SELECT
    col4,
    col5,
    col6::DATE,
    col7::DATE,
    col8
FROM temp_evenements;
```

IV – Conclusion Peuplement et Synthèse du rapport

Nous allons détailler le processus de peuplement ici et pour cela nous débutons par la création de la base de données qui accueillera les tables définies spécifiquement lors de notre première partie du rapport « Script manuel de création de la base de données ». Après s'être connecté à celle-ci via la commande **\c** suivie du nom de la base de données, nous pouvons importer les tables dans notre base de données par l'intermédiaire de la commande **\i** suivie du chemin absolu où figure le script préalablement mis dans un fichier d'extension ".sql ".

L'étape de préparation de notre plan de travail étant finie, nous pouvons nous attaquer à la véritable première étape consistant au peuplement des tables. Nous devons comme suggéré dans les consignes créer des tables temporaires qui serviront de passerelle entre les fichiers CSV et nos tables dans notre base de données. Pour cela l'utilisation de la commande " **CREATE TEMP TABLE** " suivie du nom de la table temporaire et son nombre de colonnes caractérisant le nombre de colonnes dans le fichier CSV original nous est utile .

Deuxième étape, nous devons maintenant importer les données de notre fichier CSV vers la table temporaire par l'intermédiaire de la commande **\copy** nom_table_temporaire. Cette opération nécessite une configuration précise des attributs en commençant par le format du fichier en CSV, les en-têtes des fichiers **HEADER** true.Les délimiteurs entre chaque cellule **DELIMITER ' ;** '. Il faut aussi faire attention au fichier départements-france.csv car il demande un délimiteur à ' **,** '. Enfin, le dernier attribut est celui de l'encodage qui demande la configuration spécifique L**ATIN1** car celle-ci utilise les normes ISO-8859-1.

Notre dernière étape consiste à transférer les données des tables temporaires vers les tables finales. Pour cela l'utilisation de la commande **INSERT INTO** suivie de la commande **SELECT** pour sélectionner seulement les colonnes dont les données nous sont utiles. Un point important à prendre en compte lors du peuplement via SELECT se trouve dans l'utilisation de la commande **LTRIM** , utile pour le peuplement d'id . Celle-ci permet de ne pas rencontrer de problèmes plus tard lors de connexions entre clés étrangères. En effet, des erreurs causées par des non-identification dues au fait que "05" soit différent de " 5". Elle permet par cela de généraliser nos données sous une seule forme.